

쉬운 도커

PART8. 도커 실무

TOPIC 1. 레이어 관리

TOPIC 2. 캐싱을 활용한 빌드

TOPIC 3. Nginx 설정 커스터마이징, 3 Tier 아키텍처 구성

TOPIC 4. 환경 변수를 활용한 동적 서버 설정

TOPIC 5. 컨테이너 애플리케이션 최적화

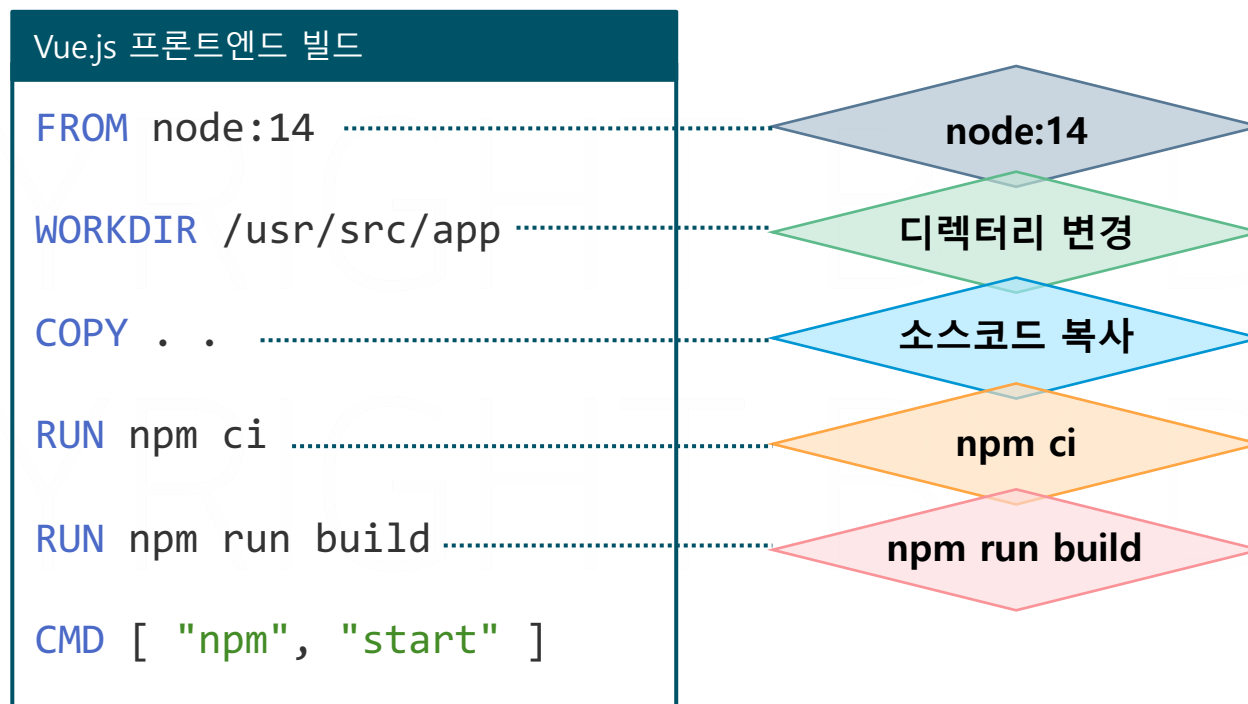
TOPIC 6. 컨테이너 내부에서 개발하기 (Node.js/VSCo

TOPIC 7. JAVA 개발환경 컨테이너 활용 (IntelliJ)

PART8. 도커 실무

TOPIC 1. 레이어 관리

- **Dockerfile**에 작성된 지시어 하나 당 레이어가 한 개 추가됩니다.
- 불필요한 레이어가 많아지면 이미지의 크기가 늘어나고 빌드 속도가 느려질 수 있습니다.



- **RUN** 지시어는 **&&**을 활용해 최대한 하나로 처리합니다.
- 불필요한 명령어를 추가해서 레이어의 개수를 늘리지 않습니다.

나쁜 예시

```
FROM ubuntu:latest
```

```
RUN apt-get update
```

```
RUN apt-get install -y curl
```

```
RUN apt-get install -y xz-utils
```

```
RUN apt-get install -y git
```

```
RUN apt-get clean
```

RUN 지시어를 5번 수행해 5개의 레이어 생성

좋은 예시

```
FROM ubuntu:latest
```

```
RUN apt-get update && \
    apt-get install -y curl && \
    apt-get install -y xz-utils && \
    apt-get install -y git && \
    apt-get clean
```

1개의 RUN 지시어 안에 5개의 명령어 삽입
1개의 레이어 생성

- **RUN** 지시어는 **&&**을 활용해 최대한 하나로 처리합니다.
- 불필요한 명령어를 추가해서 레이어의 개수를 늘리지 않습니다.

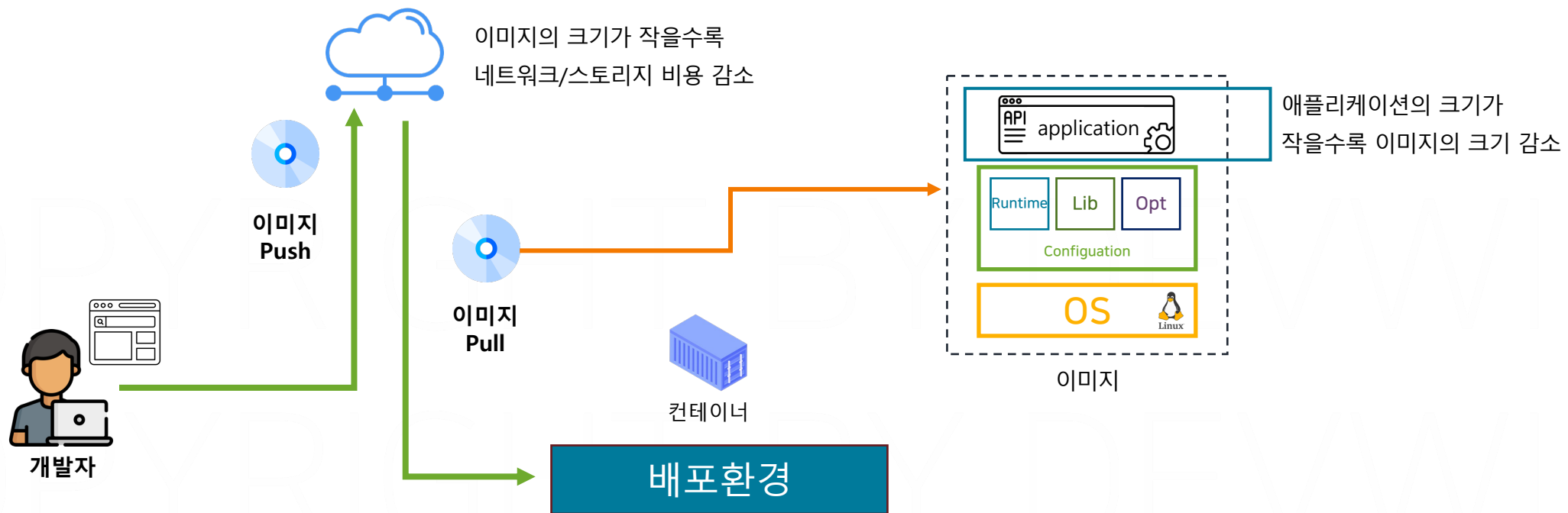
```
~/ docker image history ubuntu
```

IMAGE	CREATED	CREATED BY	SIZE
684f951cb8cd	18 hours ago	RUN /bin/sh -c apt-get clean # buildkit	0B
<missing>	18 hours ago	RUN /bin/sh -c apt-get install -y git # buil...	68.9MB
<missing>	18 hours ago	RUN /bin/sh -c apt-get install -y xz-utils #...	1.11MB
<missing>	18 hours ago	RUN /bin/sh -c apt-get install -y curl # bui...	6.7MB
<missing>	19 hours ago	RUN /bin/sh -c apt-get update # buildkit	41MB
<missing>	8 days ago	/bin/sh -c #(nop) CMD ["/bin/bash"]	0B
<missing>	8 days ago	/bin/sh -c #(nop) ADD file:8540670760767f19e...	69.2MB
<missing>	8 days ago	/bin/sh -c #(nop) LABEL org.opencontainers...	0B
<missing>	8 days ago	/bin/sh -c #(nop) LABEL org.opencontainers...	0B
<missing>	8 days ago	/bin/sh -c #(nop) ARG LAUNCHPAD_BUILD_ARCH	0B
<missing>	8 days ago	/bin/sh -c #(nop) ARG RELEASE	0B

```
~/ docker image history ubuntu
```

IMAGE	CREATED	CREATED BY	SIZE
727ea86ff6f8	18 hours ago	RUN /bin/sh -c apt-get update && apt-get...	116MB
<missing>	8 days ago	/bin/sh -c #(nop) CMD ["/bin/bash"]	0B
<missing>	8 days ago	/bin/sh -c #(nop) ADD file:8540670760767f19e...	69.2MB
<missing>	8 days ago	/bin/sh -c #(nop) LABEL org.opencontainers...	0B
<missing>	8 days ago	/bin/sh -c #(nop) LABEL org.opencontainers...	0B
<missing>	8 days ago	/bin/sh -c #(nop) ARG LAUNCHPAD_BUILD_ARCH	0B
<missing>	8 days ago	/bin/sh -c #(nop) ARG RELEASE	0B

- 애플리케이션의 크기를 가능한 작게 관리합니다.



- 애플리케이션의 크기를 가능한 작게 관리합니다.
- 베이스 이미지는 가능한 작은 이미지를 사용합니다. **alpine OS**를 사용하는 것이 좋습니다.

Ubuntu 이미지

```
FROM ubuntu:latest
RUN apt-get update && \
    apt-get install -y curl && \
    apt-get install -y xz-utils && \
    apt-get install -y git && \
    apt-get clean
```

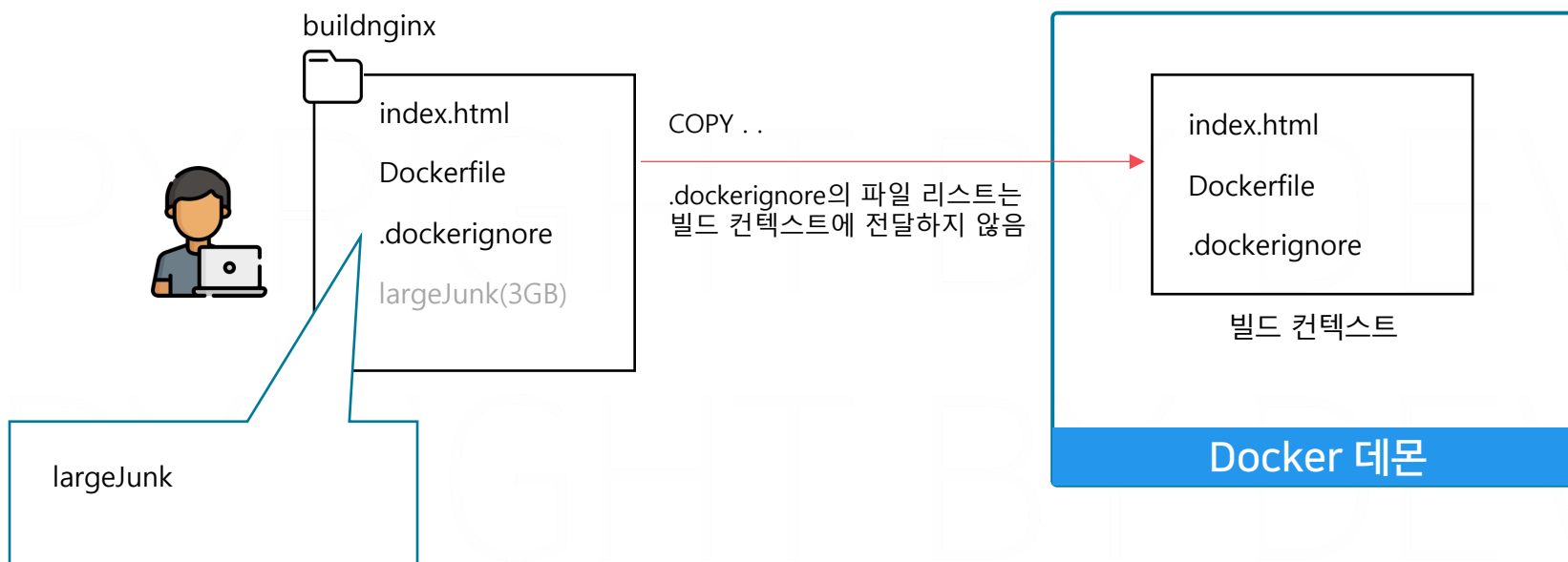
Alpine 이미지

```
FROM alpine:latest
RUN apk update && \
    apk add --no-cache curl && \
    apk add --no-cache xz && \
    apk add --no-cache git
```

```
🍏 ~/easydocker/test3/ docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
alpine	latest	ef1418452762	2 seconds ago	21.8MB
ubuntub	latest	727ea86ff6f8	19 hours ago	185MB
ubuntua	latest	684f951cb8cd	19 hours ago	187MB

- 애플리케이션의 크기를 가능한 작게 관리합니다.
- 베이스 이미지는 가능한 작은 이미지를 사용합니다. **alpine OS**를 사용하는 것이 좋습니다.
- **.dockerignore** 파일을 사용해서 불필요한 파일을 제거합니다.



- 스크래치 이미지를 사용한 빌드 실습

```
easydocker/go-scratch/main.go
```

```
package main

import (
    "fmt"
    "net/http"
)

func main() {
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintf(w, "Go, Scratch World!")
    })

    fmt.Println("Server is running on port 8080")
    http.ListenAndServe(":8080", nil)
}
```

- 스크래치 이미지를 사용한 빌드 실습

easydocker/go-scratch/Dockerfile

```
FROM golang:alpine AS builder
WORKDIR /app
COPY main.go . ----- 소스코드 복사
RUN CGO_ENABLED=0 GOOS=linux go build -a -installsuffix cgo -o helloworld main.go ----- 애플리케이션 빌드

FROM scratch
COPY --from=builder /app/helloworld . ----- 빌드스테이지에서 애플리케이션 복사
EXPOSE 8080
ENTRYPOINT ["/helloworld"] ----- 애플리케이션 실행
```

스크래치 이미지를 사용한 빌드 실습

0. 변경사항 초기화

```
git reset --hard HEAD && git clean -fd
```

1. 실습 폴더 easydocker/build 이동 및 브랜치 변경

```
git switch 02-practice
```

2. 실습 폴더 easydocker/build/05.go-scratch 이동 및 브랜치 변경

```
cd 05.go-scratch
```

3. 이미지 빌드

```
docker build -t helloworld .
```

4. 컨테이너 실행

```
docker run -d -p 8080:8080 --name go-helloworld helloworld
```

PART8. 도커 실무

TOPIC 2. 캐싱을 활용한 빌드

- **Dockerfile**에 작성된 순서대로 결과 이미지의 레이어가 쌓입니다.

node.js 애플리케이션 빌드

```
FROM node:14
```

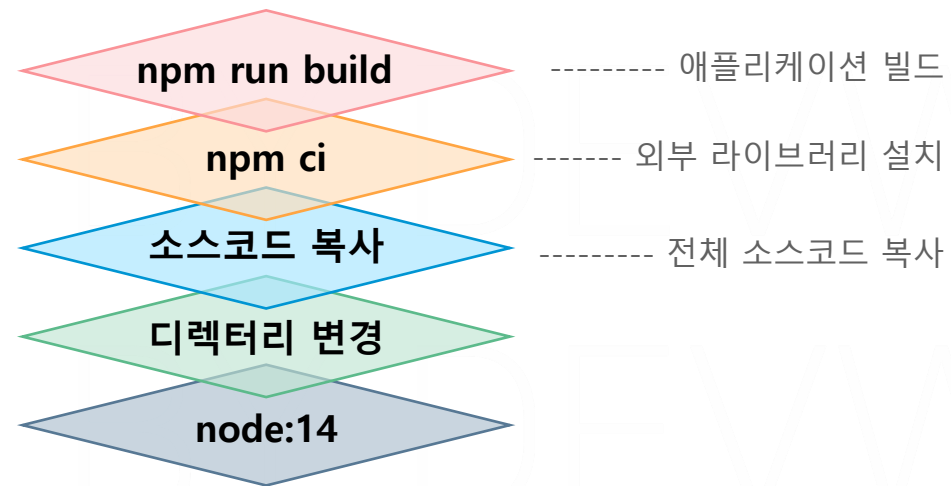
```
WORKDIR /usr/src/app
```

```
COPY . .
```

```
RUN npm ci
```

```
RUN npm run build
```

```
CMD [ "npm", "start" ]
```



1회차 docker build -t myimage .

- **Dockerfile**에 작성된 순서대로 결과 이미지의 레이어가 쌓입니다.
- Docker는 각 단계의 결과 레이어를 캐시처리합니다. 지시어가 변경되지 않으면 다음 빌드에서 레이어를 재사용합니다.

node.js 애플리케이션 빌드

```
FROM node:14
```

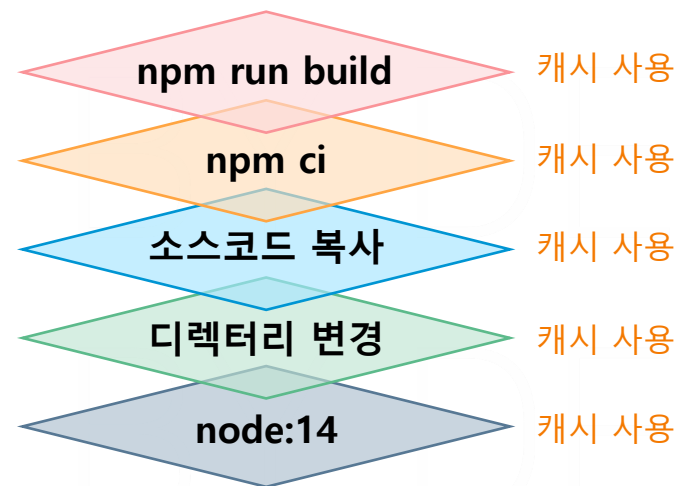
```
WORKDIR /usr/src/app
```

```
COPY . .
```

```
RUN npm ci
```

```
RUN npm run build
```

```
CMD [ "npm", "start" ]
```



2회차 docker build -t myimage .

- **Dockerfile**에 작성된 순서대로 결과 이미지의 레이어가 쌓입니다.
- Docker는 각 단계의 결과 레이어를 캐시처리합니다. 지시어가 변경되지 않으면 다음 빌드에서 레이어를 재사용합니다.
- **COPY, ADD** 명령의 경우 빌드 컨텍스트의 파일 내용이 변경되어도 캐시를 사용하지 않습니다.
- 레이어가 변경되면 그 레이어와 이후의 모든 레이어는 캐시를 사용하지 않고 새로운 레이어가 만들어집니다.

node.js 애플리케이션 빌드

```
FROM node:14
```

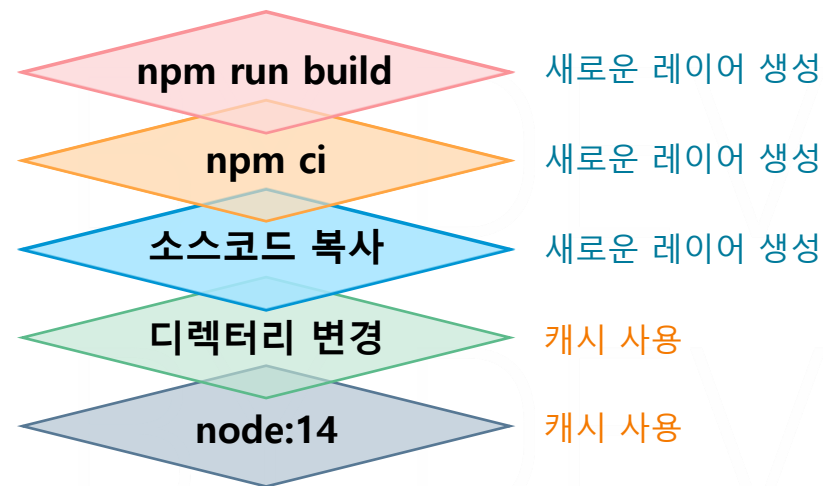
```
WORKDIR /usr/src/app
```

```
COPY . .
```

```
RUN npm ci
```

```
RUN npm run build
```

```
CMD [ "npm", "start" ]
```



소스코드 내용 변경 후
3회차 docker build -t myimage .

- 잘 변경되지 않는 파일들을 아래 레이어에 배치하면, 캐시를 활용하는 빈도를 높일 수 있습니다.
- **package.json, package-lock.json** 파일은 소스 코드가 의존하는 외부 라이브러리 정보가 있습니다. 개발 시 자주 변경되지 않습니다.
- **package.json, package-lock.json** 파일이 변경되지 않을 경우 **npm ci** 단계(의존 라이브러리 설치)까지 캐시를 활용할 수 있습니다.

캐싱을 활용한 빌드

```
FROM node:14

WORKDIR /usr/src/app

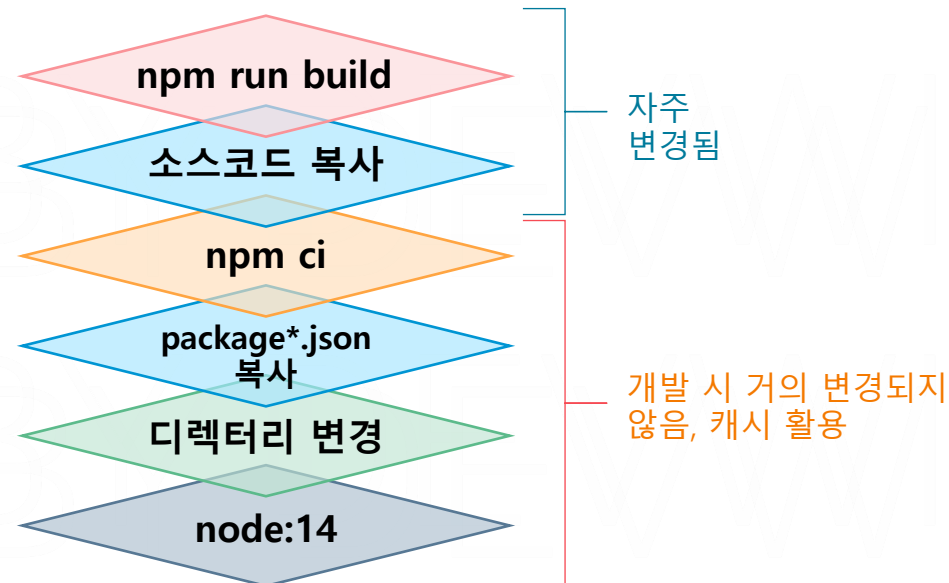
COPY package*.json ./

RUN npm ci

COPY . .

RUN npm run build

CMD [ "npm", "start" ]
```



leafy-frontend 도커파일 수정 및 캐싱 테스트

1. easydocker/leafy/leafy-frontend 디렉터리 이동 및 소스코드 상태 변경

```
git reset --hard HEAD && git clean -fd
```

```
git switch 01-dockerfile
```

2. 코드 변경 없이 2번 빌드 시도, 2번째 빌드에서 캐시 사용 확인

```
docker build -t leafy-front:2.0.0 . --no-cache
```

```
docker build -t leafy-front:2.0.1 .
```

3. 코드 일부분 수정 후 다시 빌드 시도, 시간 체크

(src/App.vue)

```
docker build -t leafy-front:2.0.2 .
```

4. 다음 페이지의 도커파일을 참고해 도커파일 수정 및 빌드

```
docker build -t leafy-front:2.0.3 . --no-cache
```

5. 코드 일부분 수정 후 다시 빌드 시도, 캐시 활용 한 시간 체크

```
docker build -t leafy-front:2.0.4 .
```

```
easydocker/leafy/leafy-frontend/Dockerfile
```

```
FROM node:14 AS build
```

```
WORKDIR /app
```

```
COPY package.json .
COPY package-lock.json .
```

----- 라이브러리 설치에 필요한 package.json 파일과 package-lock.json 파일만 복사

```
RUN npm ci
```

----- 의존 라이브러리 설치

```
COPY . /app
```

----- 전체 소스코드 복사

```
RUN npm run build
```

----- 애플리케이션 빌드

```
FROM nginx:1.21.4-alpine
```

```
COPY --from=build /app/dist /usr/share/nginx/html
```

```
EXPOSE 80
```

```
ENTRYPOINT ["nginx"]
```

```
CMD ["-g", "daemon off;"]
```

leafy-backend 도커파일 수정 및 캐싱 테스트

1. easydocker/leafy/leafy-backend 이동 및 다음 파일의 도커파일 수정 후 빌드

```
cd ../leafy-backend
```

```
docker build -t leafy-backend:2.0.0 . --no-cache
```

2. 소스파일 변경 후 재빌드,

(src/main/java/com/devwiki/leafy/controller/plant/PlantController.java)

캐시 활용하여 단축된 시간 확인

```
docker build -t leafy-backend:2.0.1 .
```

easydocker/leafy/leafy-backend/Dockerfile

```
FROM gradle7.6.1-jdk11 AS build
```

```
WORKDIR /app
```

```
COPY build.gradle settings.gradle ./
```

----- 라이브러리 설치에 필요한 build.gradle파일과
settings.gradle 파일만 복사

```
RUN gradle dependencies --no-daemon
```

----- 의존 라이브러리 설치

```
COPY . /app
```

----- 전체 소스코드 복사

```
RUN gradle clean build --no-daemon
```

----- 애플리케이션 빌드

```
FROM openjdk:11-jre-slim
```

```
WORKDIR /app
```

```
COPY --from=build /app/build/libs/*.jar /app/leafy.jar
```

```
EXPOSE 8080
```

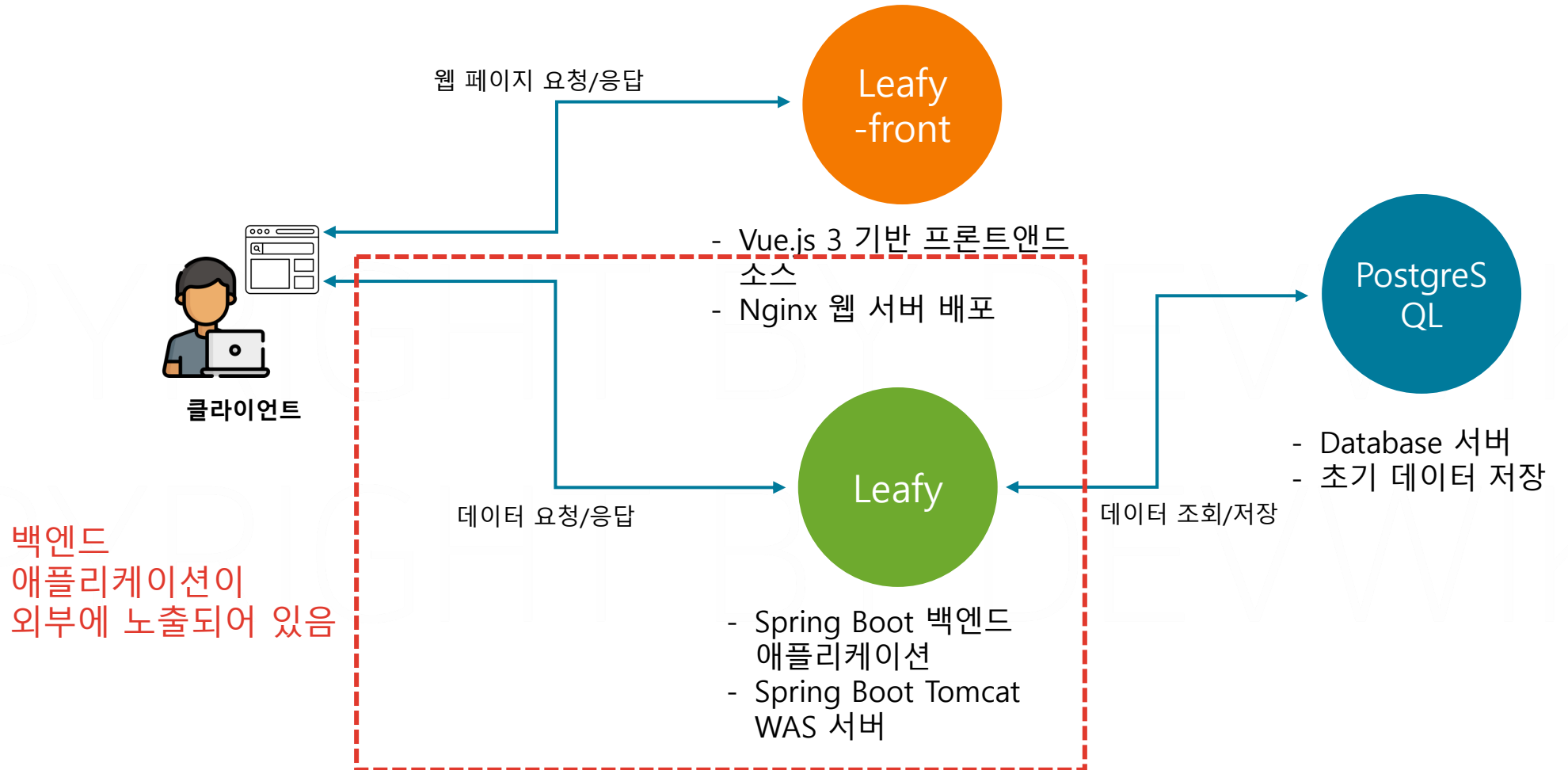
```
ENTRYPOINT ["java"]
```

```
CMD ["-jar", "leafy.jar"]
```

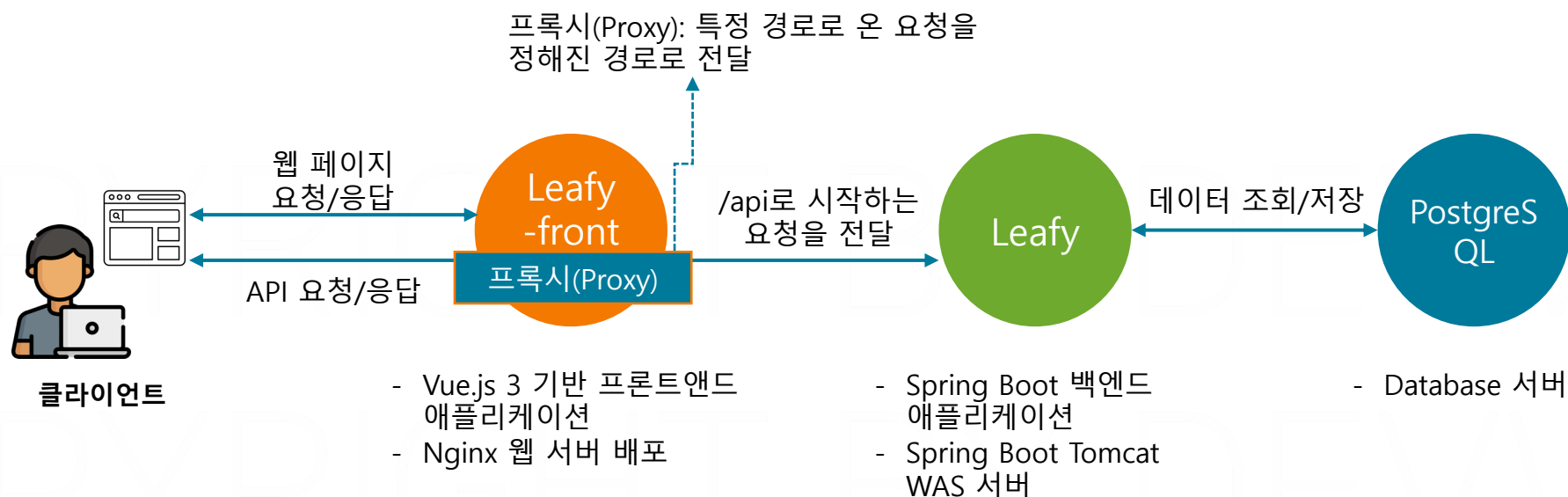
PART8. 도커 실무

TOPIC 3. 3 Tier 아키텍처 구성

- 백엔드 애플리케이션이 외부에 노출되어 있을 경우 개발자가 의도하지 않은 API를 호출할 위험이 있습니다.



- Nginx의 프록시 기술을 활용해 보안에 뛰어난 3Tier 아키텍처를 구성할 수 있습니다.
- Nginx는 특정 경로로 온 요청(/api로 시작하는 경로)를 지정한 서버로 전달합니다.
- Nginx를 프록시 서버로 활용하여 보안 향상, 부하 관리 및 API 응답 개성을 활용할 수 있습니다.



Nginx 서버 설정 변경

```
location /api/ {
    proxy_pass http://leafy:8080;
}
```

leafy-frontend 도커파일 수정 및 캐싱 테스트

1. easydocker/leafy/leafy-frontend 디렉터리 이동 및 소스코드 상태 변경

```
cd ../leafy-frontend
```

```
git reset --hard HEAD && git clean -fd
```

```
git switch 02-cache
```

또는

```
git switch 03-proxy
```

easydocker/leafy/leafy-frontend/src/api/api.js 파일 수정

```
import axios from 'axios';
```

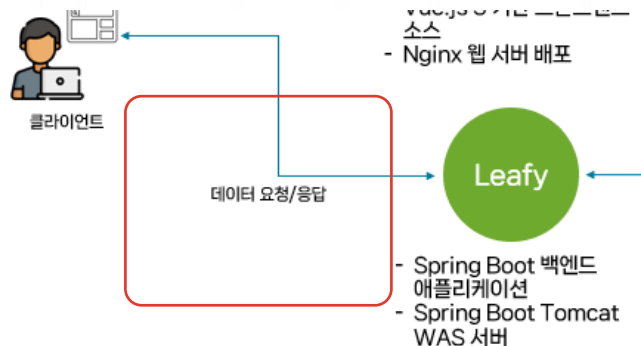
```
const api = axios.create({
```

```
  baseUrl: process.env.VUE_APP_API_BASE_URL || 'http://localhost:8080'
```

```
});
```

```
export default api;
```

웹 브라우저에서 백엔드 애플리케이션에 대한 요청을 http://localhost:8080으로 보내는 설정
해당 라인 제거 후 저장



easydocker/leafy/leafy-frontend/nginx.conf 파일 생성

```
server {  
    listen      80;  
    server_name _;  
  
    location / {  
        root    /usr/share/nginx/html;  
        index   index.html index.htm;  
    }  
  
    location /api/ {  
        proxy_pass http://leafy:8080;  
    }  
  
    error_page   500 502 503 504   /50x.html;  
    location = /50x.html {  
        root    /usr/share/nginx/html;  
    }  
}
```

----- 웹 서버로 오는 요청 중
/api/로 시작하는 경로의 요청을
모두 http://leafy:8080으로 전달

easydocker/leafy/leafy-frontend/Dockerfile

```
FROM node:14 AS build
WORKDIR /app
COPY package.json .
COPY package-lock.json .
RUN npm ci
COPY . .
RUN npm run build
```

```
FROM nginx:1.21.4-alpine
```

```
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

소스코드의 nginx.conf 파일을 이미지 빌드 시
nginx 설정으로 복사

```
COPY --from=build /app/dist /usr/share/nginx/html
```

```
EXPOSE 80
```

```
ENTRYPOINT ["nginx"]
```

```
CMD ["-g", "daemon off;"]
```

leafy-frontend 이미지 빌드 및 애플리케이션 구동 테스트

1. 이미지 빌드

```
docker build -t leafy-front:3.0.0-proxy .
```

2. network 가 제대로 생성되어 있는지 확인

```
docker network create leafy-network
```

3. 애플리케이션 구동 및 테스트, 백엔드 애플리케이션을 포트포워딩 하지 않고 내부 통신만 사용하도록 설정

```
(MacOS) docker run -d --name leafy-postgres -v mydata:/var/lib/postgresql/data --network leafy-network devwikirepo/leafy-postgres:1.0.0
```

```
(Windows) docker run -d --name leafy-postgres -v mydata://var/lib/postgresql/data --network leafy-network devwikirepo/leafy-postgres:1.0.0
```

```
docker run -d -e DB_URL=leafy-postgres --name leafy --network leafy-network devwikirepo/leafy-backend:1.0.0
```

```
docker run -d -p 80:80 --name leafy-front --network leafy-network leafy-front:3.0.0-proxy
```

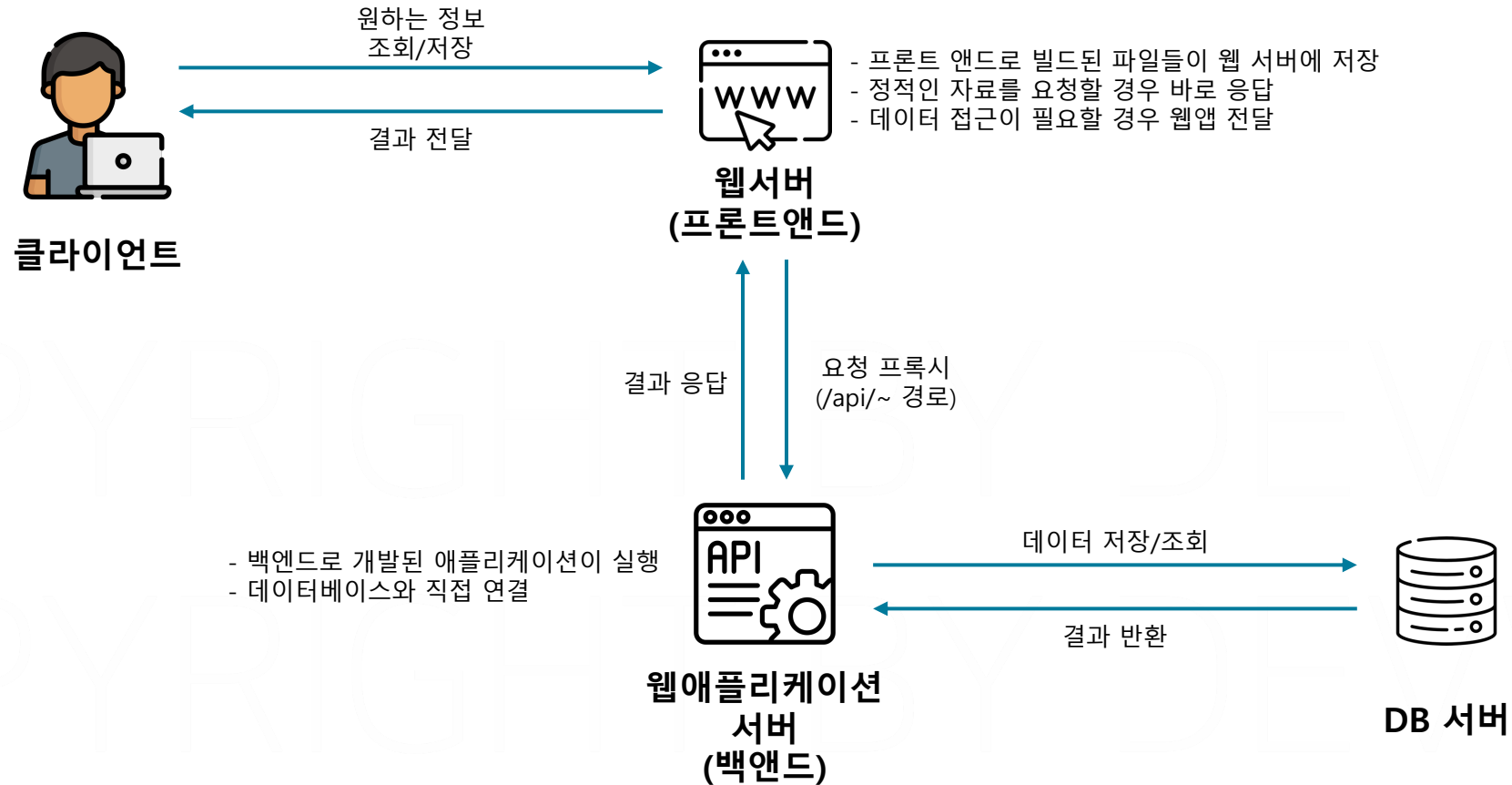
4. localhost:80 접속하여 정상 접속 확인, curl 명령으로 백엔드 접근 테스트(응답이 오지 않아야 정상)

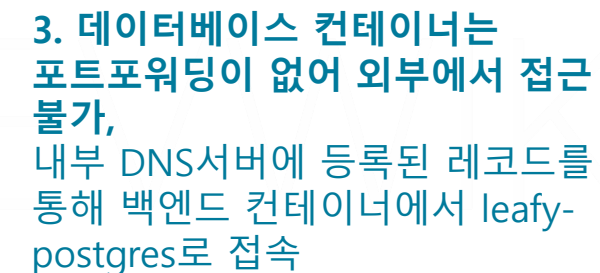
```
curl http://localhost:8080/api/v1/users
```

5. 환경 정리

```
docker rm -f leafy-postgres leafy leafy-front
```

```
docker volume rm mydata
```





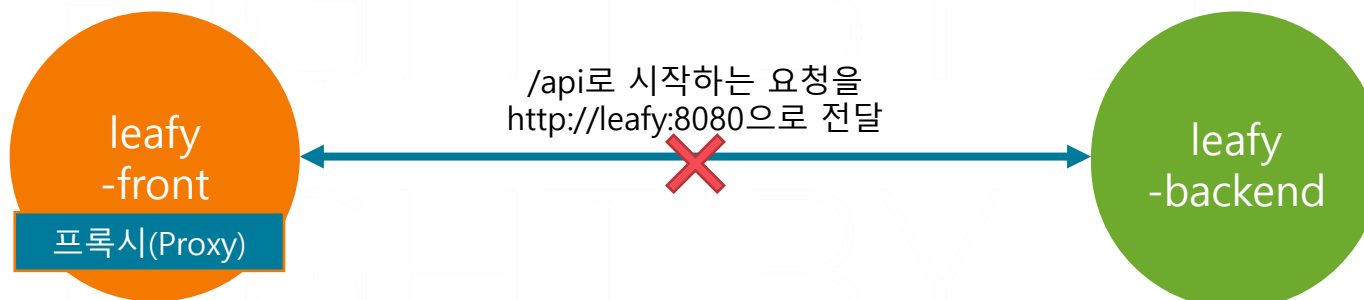
PART8. 도커 실무

TOPIC 4. 동적 서버 설정

- Nginx 서버 설정에 백엔드 애플리케이션의 주소가 고정되어 있습니다.
- 환경 별로 Nginx가 프록시 해야 하는 주소가 바뀔 수 있습니다.
- 프록시 설정의 주소를 바꾸기 위해 이미지 빌드를 다시 해야 합니다.

Nginx 서버 프록시 설정

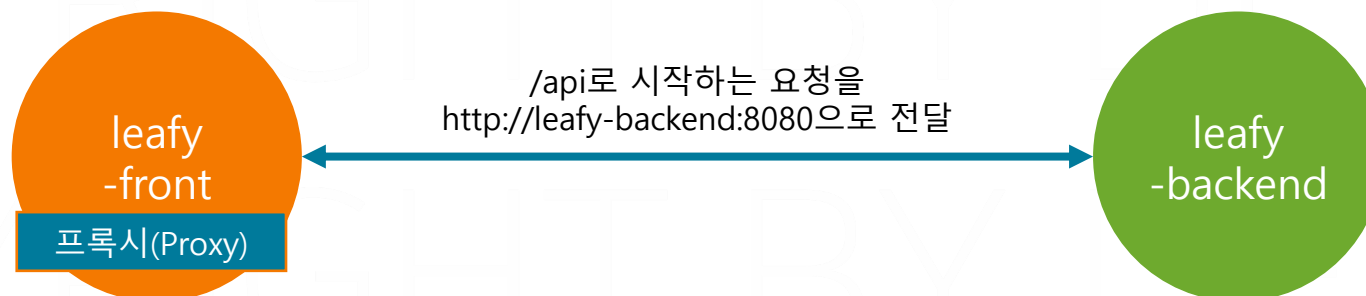
```
location /api/ {  
    proxy_pass http://leafy:8080;  
}
```



- 환경 별로 달라지는 정보는 시스템 환경 변수로 처리하면 컨테이너 실행 시 결정할 수 있습니다.

Nginx 서버 프록시 설정

```
location /api/ {  
    proxy_pass http://${BACKEND_HOST}:${BACKEND_PORT};  
}
```



Nginx 실행 시 환경 변수 지정

```
BACKEND_HOST = leafy-backend  
BACKEND_PORT = 8080
```

leafy-frontend 도커파일 수정 및 캐싱 테스트

1. easydocker/leafy/leafy-frontend 디렉터리 이동 및 소스코드 상태 변경

```
git reset --hard HEAD && git clean -fd
```

```
git switch 03-proxy
```

또는

```
git switch 04-dynamicconfig
```

easydocker/leafy/leafy-frontend/nginx.conf 파일 수정

```
server {  
    listen      80;  
    server_name localhost;  
  
    location / {  
        root    /usr/share/nginx/html;  
        index  index.html index.htm;  
    }  
  
    location /api/ {  
        proxy_pass http://${BACKEND_HOST}:${BACKEND_PORT};  
    }  
  
    error_page 500 502 503 504 /50x.html;  
    location = /50x.html {  
        root    /usr/share/nginx/html;  
    }  
}
```

----- 웹 서버로 오는 요청 중
/api/로 시작하는 경로의 요청을
http://\${BACKEND_HOST}:\${BACKEND_PORT}으로 전달

easydocker/leafy/leafy-frontend/Dockerfile 수정

```
FROM node:14 AS build
WORKDIR /app
COPY package.json .
COPY package-lock.json .
RUN npm ci
COPY . .
RUN npm run build
```

```
FROM nginx:1.21.4-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf.template
ENV BACKEND_HOST leafy
ENV BACKEND_PORT 8080
```

```
COPY docker-entrypoint.sh /usr/local/bin/
RUN chmod +x /usr/local/bin/docker-entrypoint.sh
```

```
COPY --from=build /app/dist /usr/share/nginx/html
```

```
EXPOSE 80
```

```
ENTRYPOINT ["docker-entrypoint.sh"]
CMD ["nginx", "-g", "daemon off;"]
```

----- 소스코드의 nginx.conf 파일을
template 파일로 복사
기본 환경 변수 지정

-- 컨테이너 실행시 자동으로 실행될
스크립트 지정

----- 스크립트 실행 처리

easydocker/leafy/leafy-frontend/docker-entrypoint.sh 생성

```
#!/bin/sh
```

```
set -e
```

```
# default.conf.template 파일에서 환경 변수를 대체하고 결과를 default.conf에 저장
```

```
envsubst '${BACKEND_HOST} ${BACKEND_PORT}' < /etc/nginx/conf.d/default.conf.template > /etc/nginx/conf.d/default.conf
```

```
# 다음 명령어를 실행
```

```
exec "$@"
```

템플릿 파일과 환경 변수를 읽은 뒤
nginx.conf 파일을 구성하여 nginx 설정으로 복사

leafy-frontend 이미지 빌드 및 애플리케이션 구동 테스트

1. 이미지 빌드

```
docker build -t leafy-front:4.0.0-env .
```

2. network 가 제대로 생성되어 있는지 확인

```
docker network create leafy-network
```

3. 애플리케이션 구동 및 테스트, 백엔드 애플리케이션의 접속 도메인 변경

localhost:80 접속하여 정상 접속 확인

(MacOS) `docker run -d --name leafy-postgres -v mydata:/var/lib/postgresql/data --network leafy-network devwikirepo/leafy-postgres:1.0.0`

(Windows) `docker run -d --name leafy-postgres -v mydata://var/lib/postgresql/data --network leafy-network devwikirepo/leafy-postgres:1.0.0`

```
docker run -d -e DB_URL=leafy-postgres --name leafy-backend --network leafy-network devwikirepo/leafy-backend:1.0.0
```

```
docker run -d -e BACKEND_HOST=leafy-backend -p 80:80 --name leafy-front --network leafy-network leafy-front:4.0.0-env
```

4. 설정 파일에 환경 변수가 적용되었는지 확인

```
docker exec leafy-front cat etc/nginx/conf.d/default.conf
```

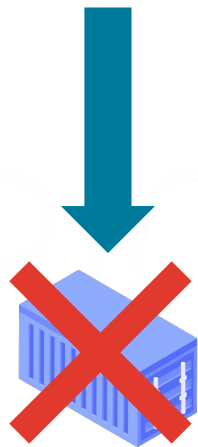
5. 환경 정리

```
docker rm -f leafy-postgres leafy-backend leafy-front
```

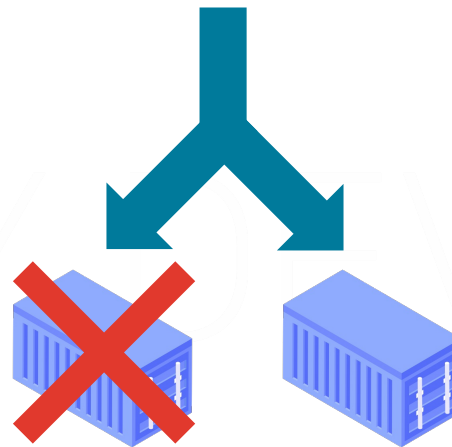
PART8. 도커 실무

TOPIC 5. PostgreSQL 이중화 DB 구성

- 단일 서버 구성 시 단일 서버에 장애가 생기면 전체 서비스의 장애로 이어집니다.
- 서버 이중화(Redundancy) 구성 시 하나의 서버가 실패해도 다른 서버가 동일한 역할을 수행하여 **고가용성**을 보장합니다.

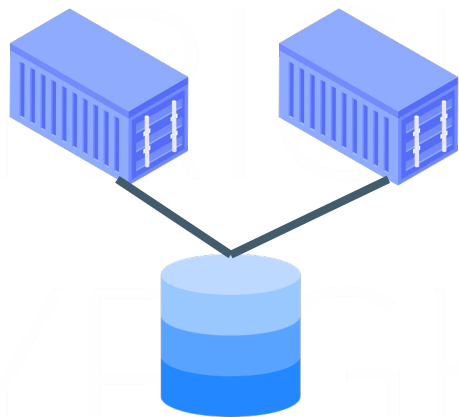


단일 서버 구성

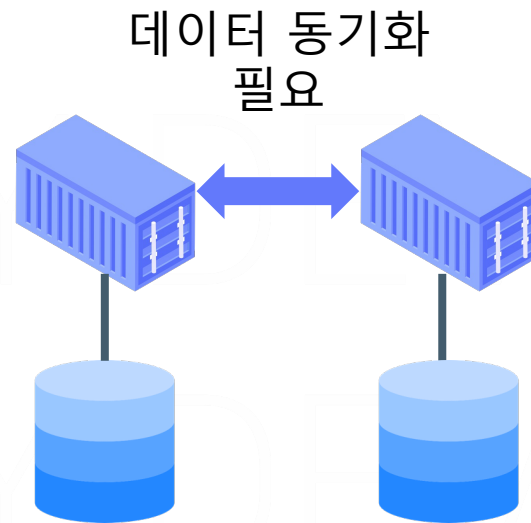


서버 이중화 구성

- 동시에 같은 볼륨을 사용하거나, 각각의 컨테이너에 별도의 볼륨을 연결할 수 있습니다.
- 동시에 같은 볼륨을 사용하면 구성이 간단하지만 볼륨에 문제가 생길 경우 대처가 어렵습니다.
- 동시에 같은 볼륨을 사용하면 볼륨의 성능에 부하가 생길 수 있습니다.
- 각각의 컨테이너에 별도의 볼륨을 연결하면 데이터의 싱크를 맞추는 처리를 별도로 해야 합니다.

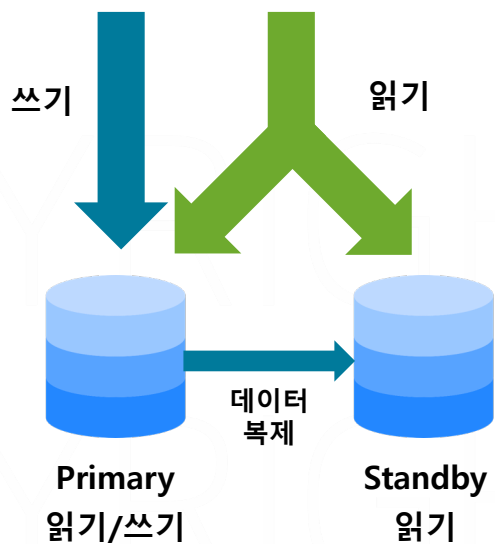


동시에 같은 볼륨 사용

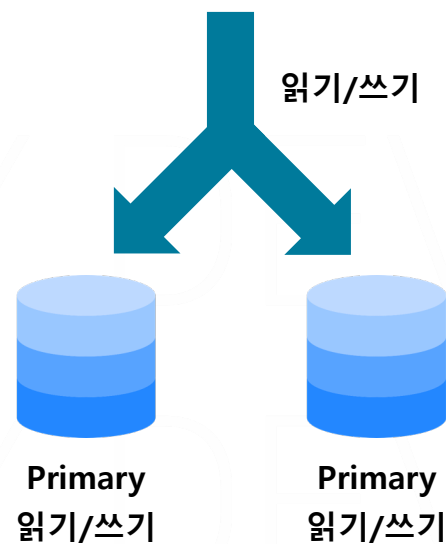


별도의 볼륨 사용

- 프라이머리-스탠바이 복제 구조의 경우 프라이머리 서버에만 쓰기 작업을 수행하며, 프라이머리의 상태를 스탠바이에 복제합니다. 스탠바이 서버는 읽기 전용으로만 사용되며, 읽기 전용 스탠바이 서버를 여러 대 사용할 수 있습니다.
- 프라이머리- 프라이머리 복제 구조의 경우 모든 서버에 읽기/쓰기 작업을 수행합니다. 여러 서버에서 동시에 쓰기 작업이 일어나기 때문에 동기화 구성 작업이 복잡합니다.



Primary-Standby Replication



Primary-Primary Replication

프라이머리-스탠바이 이중화 Postgres 구성 (프라이머리 노드)

1. easydocker/leafy/leafy-postgres 디렉터리 이동 및 소스코드 상태 변경

```
git reset --hard HEAD && git clean -fd
```

```
git switch 05-redundancy
```

2. 테스트용 네트워크 생성

```
docker network create postgres
```

COPYRIGHT BY DEVWIKI

COPYRIGHT BY DEVWIKI

프라이머리-스탠바이 이중화 Postgres 구성 (프라이머리 노드)

3. 프라이머리 노드 실행

```
docker run -d \  
  --name postgres-primary-0 \  
  --network postgres \ Windows는 //bitnami/postgresql  
  -v postgres_primary_data:/bitnami/postgresql \  
  -e POSTGRESQL_POSTGRES_PASSWORD=adminpassword \ ----- postgres 사용자(슈퍼유저)의 비밀번호  
  -e POSTGRESQL_USERNAME=myuser \ ----- 데이터베이스 사용자 이름  
  -e POSTGRESQL_PASSWORD=mypassword \ ----- 데이터베이스 사용자 패스워드  
  -e POSTGRESQL_DATABASE=mydb \ ----- 데이터베이스 명  
  -e REPMGR_PASSWORD=repmgrpassword \ ----- Repmgr 패스워드(레플리케이션 관리용)  
  -e REPMGR_PRIMARY_HOST=postgres-primary-0 \ ----- 프라이머리 노드의 호스트 명  
  -e REPMGR_PRIMARY_PORT=5432 \ ----- 프라이머리 노드의 포트  
  -e REPMGR_PARTNER_NODES=postgres-primary-0,postgres-standby-1:5432 \ ----- 통신할 노드 목록  
  -e REPMGR_NODE_NAME=postgres-primary-0 \ ----- 현재 노드의 이름  
  -e REPMGR_NODE_NETWORK_NAME=postgres-primary-0 \ ----- 현재 노드의 도메인명  
  -e REPMGR_PORT_NUMBER=5432 \ ----- 현재 노드의 포트  
  bitnami/postgresql-repmgr:15
```

프라이머리-스탠바이 이중화 Postgres 구성 (스탠바이 노트)

4. 스탠바이 노트 실행

```
docker run -d \  
  --name postgres-standby-1 \  
  --network postgres \ Windows는 //bitnami/postgresql \  
  -v postgres_standby_data:/bitnami/postgresql \  
  -e POSTGRESQL_POSTGRES_PASSWORD=adminpassword \  
  -e POSTGRESQL_USERNAME=myuser \  
  -e POSTGRESQL_PASSWORD=mypassword \  
  -e POSTGRESQL_DATABASE=mydb \  
  -e REPMGR_PASSWORD=repmgrpassword \  
  -e REPMGR_PRIMARY_HOST=postgres-primary-0 \  
  -e REPMGR_PRIMARY_PORT=5432 \  
  -e REPMGR_PARTNER_NODES=postgres-primary-0,postgres-standby-1:5432 \  
  -e REPMGR_NODE_NAME=postgres-standby-1 \  
  -e REPMGR_NODE_NETWORK_NAME=postgres-standby-1 \  
  -e REPMGR_PORT_NUMBER=5432 \  
  bitnami/postgresql-repmgr:15
```

프라이머리-스탠바이 이중화 Postgres 구성 (데이터 테스트)

5. SHELL1, SHELL2 각 컨테이너의 로그 확인

```
docker logs -f postgres-primary-0
```

```
docker logs -f postgres-standby-1
```

6. 프라이머리 노드에 테이블 생성 및 데이터 삽입

```
docker exec -it -e PGPASSWORD=mypassword postgres-primary-0 psql -U myuser -d mydb -c "CREATE TABLE sample (id SERIAL PRIMARY KEY, name VARCHAR(255));"
```

```
docker exec -it -e PGPASSWORD=mypassword postgres-primary-0 psql -U myuser -d mydb -c "INSERT INTO sample (name) VALUES ('John'), ('Jane'), ('Alice');"
```

7. 스탠바이 노드에 데이터가 동기화되어 있는지 확인

```
docker exec -it -e PGPASSWORD=mypassword postgres-standby-1 psql -U myuser -d mydb -c "SELECT * FROM sample;"
```

8. 환경 정리

```
docker rm -f postgres-primary-0 postgres-standby-1
```

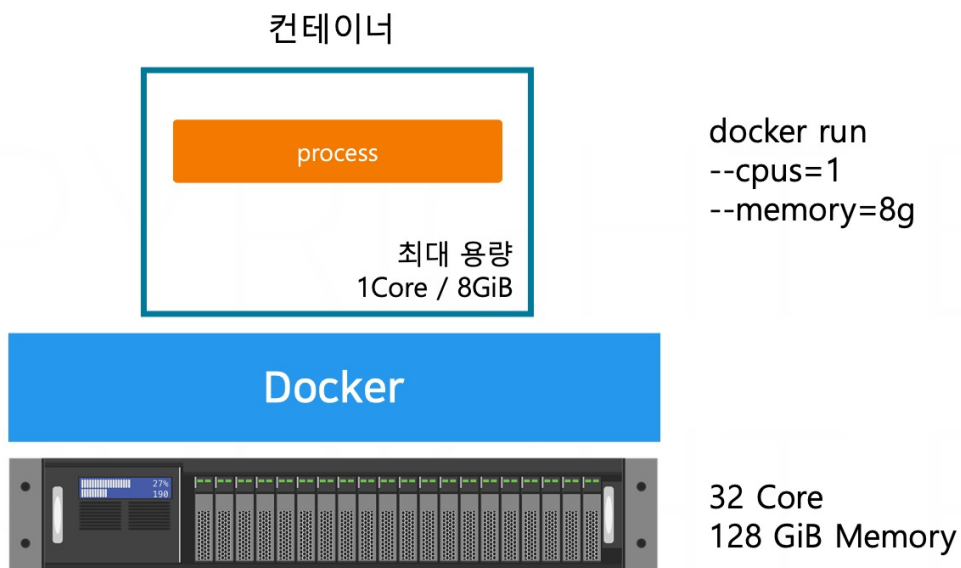
```
docker volume rm postgres_primary_data postgres_standby_data
```

```
docker network rm postgres
```

PART8. 도커 실무

TOPIC 6. 컨테이너 애플리케이션 최적화

1. 컨테이너가 사용할 수 있는 리소스 사용량을 제한



docker run

--cpus={CPUcore수}컨테이너가 사용할 최대 CPU코어 수 정의
(호스트 머신의 CPU 성능에 상대적)**--memory={메모리용량}**컨테이너가 사용할 최대 메모리 정의
(b, k, m, g 단위로 지정 가능)

docker stats (컨테이너명/ID)

컨테이너의 리소스 사용량 조회

docker events

HOSTOS에서 발생하는 이벤트 로그 조회

1. 컨테이너가 사용할 수 있는 리소스 사용량을 제한

nginx 구동 시 리소스 사용량 제한

1. 리소스 제약이 없는 상태로 컨테이너 실행

```
docker run --help
```

```
docker run -d --name no-limit nginx
```

2. 컨테이너의 메타데이터 확인

```
docker inspect no-limit | grep -e Memory -e Cpus
```

3. 리소스 제약이 있는 상태로 컨테이너 실행 (0.5 Core / 256M Memory)

```
docker run -d --name with-limit --cpus=0.5 --memory=256M nginx
```

4. 컨테이너의 메타데이터 확인

```
docker inspect no-limit | grep -e Memory -e Cpus
```

5. 실습 컨테이너 삭제

```
docker rm -f no-limit with-limit
```

1. 컨테이너가 사용할 수 있는 리소스 사용량을 제한

LIMIT에 지정한 CPU보다 사용량이 초과할 경우 CPU 스로틀링 발생

- 컨테이너에 설정된 CPU LIMIT을 초과하는 CPU 사용이 감지되면, 시스템은 컨테이너의 CPU 사용을 제한
- 애플리케이션의 성능 저하 발생

LIMIT에 지정한 MEMORY보다 사용량이 초과할 경우

- OOM(Out Of Memory) Killer 프로세스가 실행되고 컨테이너가 강제로 종료

1. 컨테이너가 사용할 수 있는 리소스 사용량을 제한

메모리 사용량이 초과될 경우 에러 확인

2. 리소스 사용량이 10MB 이상을 사용하는 컨테이너 실행

```
docker run -d --name memoryuse devwikirepo/memoryuse
```

```
docker stats
```

3. 동일한 컨테이너로 10MB로 리소스 사용량 제약을 준 후 실행

```
docker run -d --name memoryuse-withlimit --memory=10M devwikirepo/memoryuse
```

4. 컨테이너의 상태 조회

```
docker ps
```

5. 실습 컨테이너 삭제

```
docker rm -sf memoryuse-withlimit memoryuse with-limit no-limits
```

컨테이너의 상태 확인

1. 이벤트 로그 조회

```
docker events
```

====> 실습 종료

1. 컨테이너가 사용할 수 있는 리소스 사용량을 제한

leafy 애플리케이션 구동 시 리소스 사용량 제한

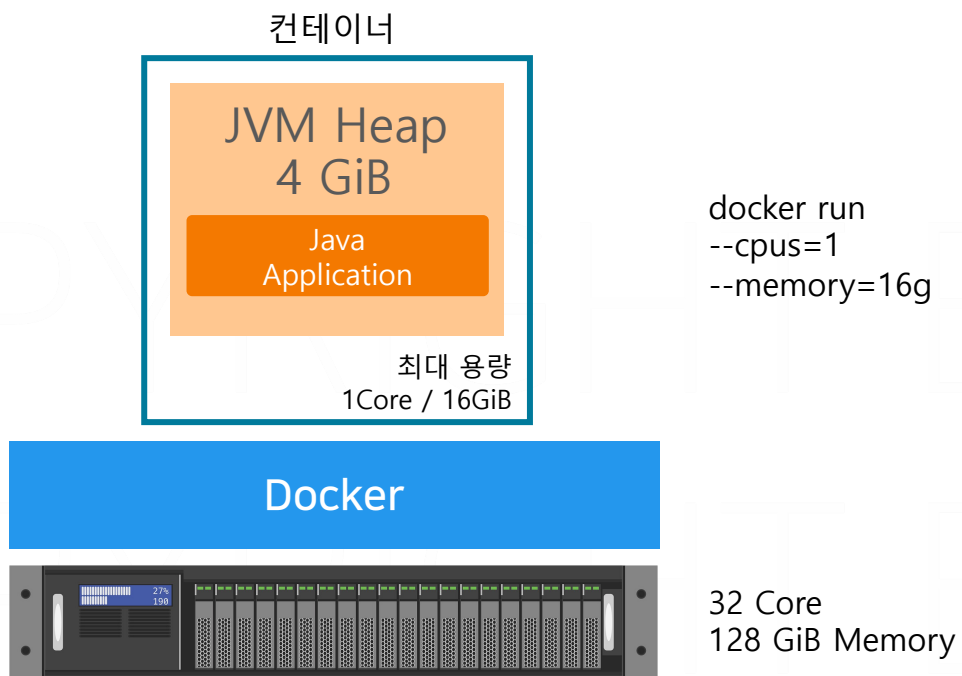
```
docker run -d --cpus=1 --memory=256m --name leafy-postgres -v mydata:/var/lib/postgresql/data --network leafy-network devwikirepo/leafy-postgres:1.0.0
docker run -d --cpus=1.5 --memory=512m -e DB_URL=leafy-postgres --name leafy --network leafy-network devwikirepo/leafy-backend:1.0.0
docker run -d --cpus=0.5 --memory=64m -e BACKEND_HOST=leafy -p 80:80 --name leafy-front --network leafy-network leafy-front:4.0.0-env
```

COPYRIGHT BY DEVWIK

COPYRIGHT BY DEVWIK

2. 자바 가상 머신(JVM) 튜닝

- JVM(Java Virtual Machine)은 자바를 실행할 수 있는 환경입니다.
- 자바 애플리케이션이 사용할 수 있는 메모리 영역인 힙(Heap) 메모리를 별도로 관리해야 합니다.



2. 자바 가상 머신(JVM) 튜닝

- JVM(Java Virtual Machine)은 자바를 실행할 수 있는 환경입니다.
- JRE(Java Runtime Environment)에서 `java -jar` 명령어로 `app.jar` 파일을 실행 시킬 때 JVM 위에서 애플리케이션이 실행되게 됩니다. 따라서 JVM 설정을 올바르게 세팅해야 합니다.

easydocker/leafy/leafy-backend/Dockerfile

... (빌드 부분 생략)

FROM openjdk:8-jre-alpine

JVM 튜닝을 위한 환경 변수 추가

ENV JAVA_OPTS="-XX:+UnlockExperimentalVMOptions -XX:+UseCGroupMemoryLimitForHeap"

WORKDIR /app

COPY --from=build /app/build/libs/*.jar /app/leafy.jar

EXPOSE 8080

ENTRYPOINT ["java"]

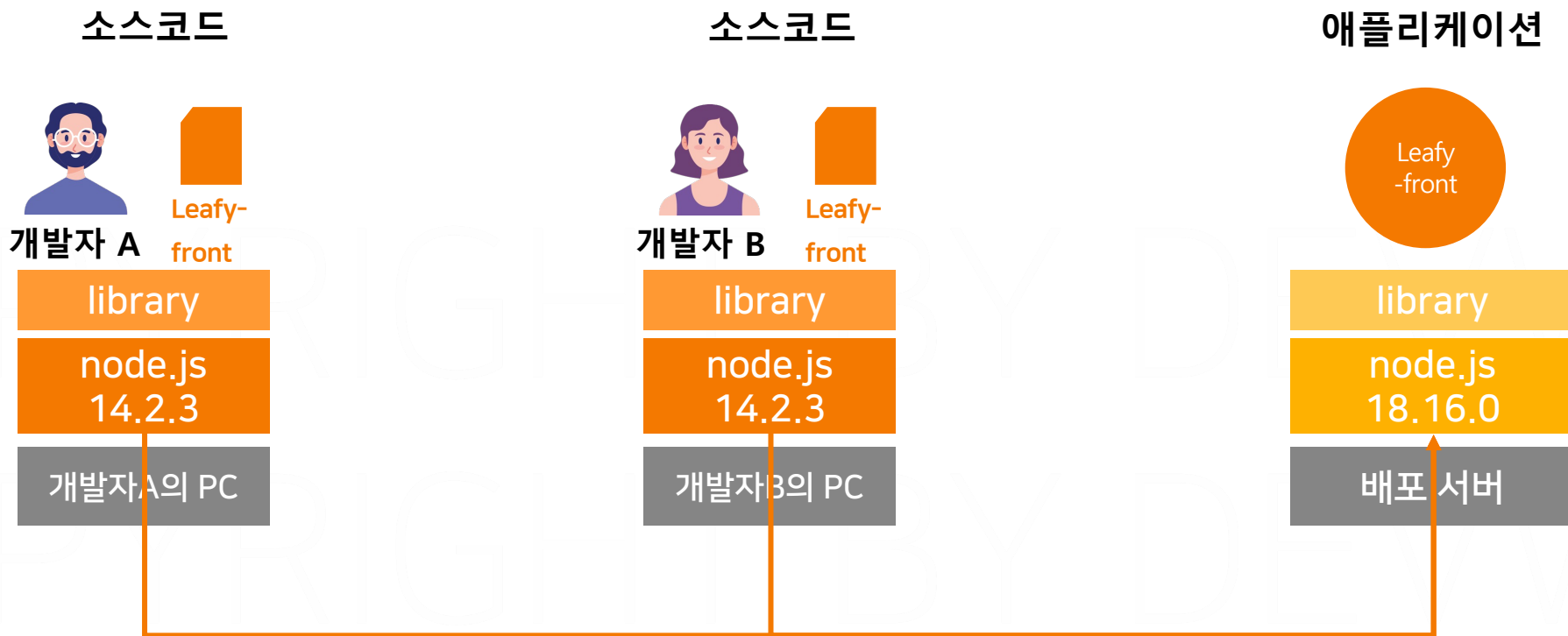
CMD ["-jar", "leafy.jar"]

JVM의 Heap 메모리를 컨테이너에 할당된 메모리에 맞추어 자동으로 조절합니다.
(Java 8u131 부터 지원, Java 10 이상에서는 기본 활성화)

PART8. 도커 실무

TOPIC 7. 컨테이너 내부에서 개발하기
(Node.js / VSCode)

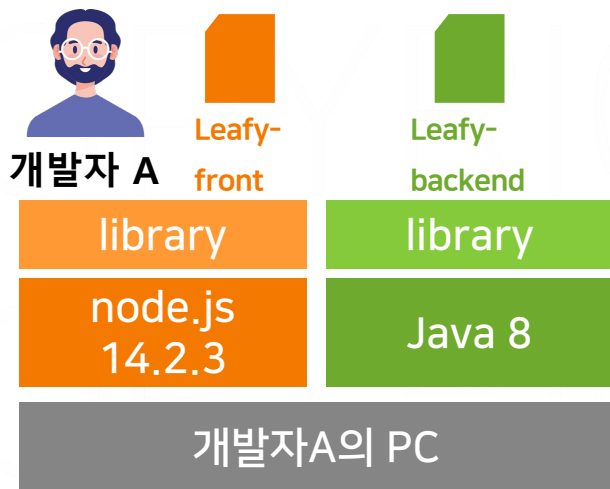
- 개발자의 PC마다 설치된 언어 및 라이브러리의 버전을 일치시키기 쉽지 않습니다.



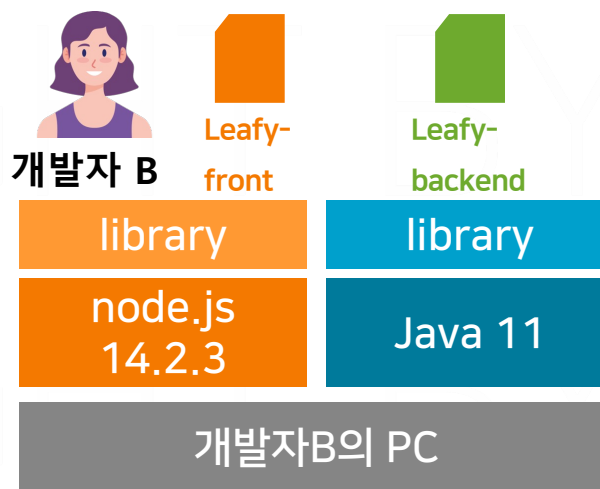
개발 환경과 배포 환경이 다를 경우 배포 시 장애 발생

- 개발자의 PC마다 설치된 언어 및 라이브러리의 버전을 일치시키기 쉽지 않습니다.
- 새로운 개발자가 합류하면 개발 환경 설정에 오랜 시간이 걸리고, 개발 환경 버전이 변화하면 싱크를 맞추기 어려워 집니다.

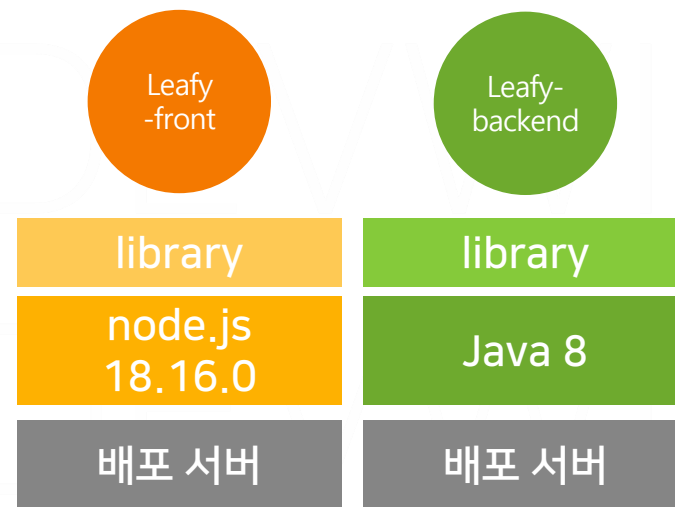
소스코드



소스코드

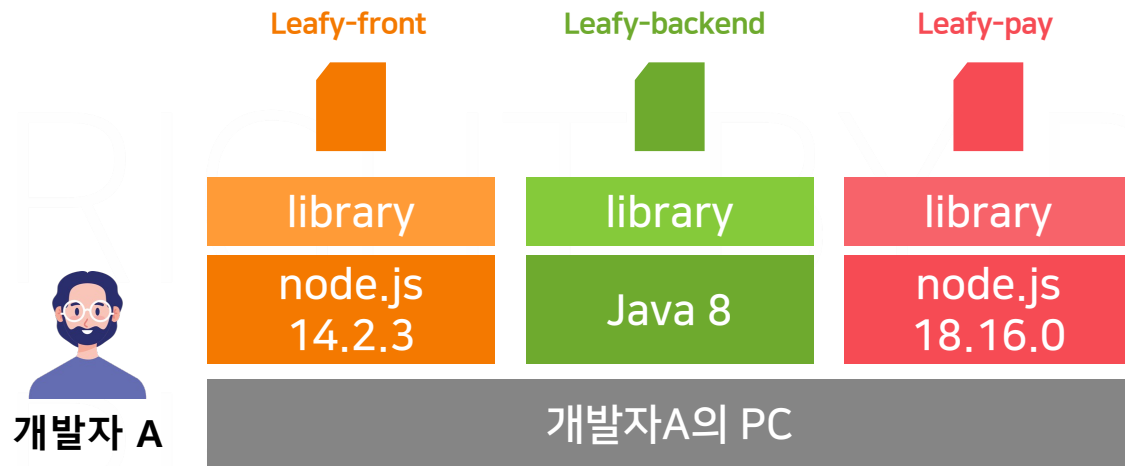


애플리케이션

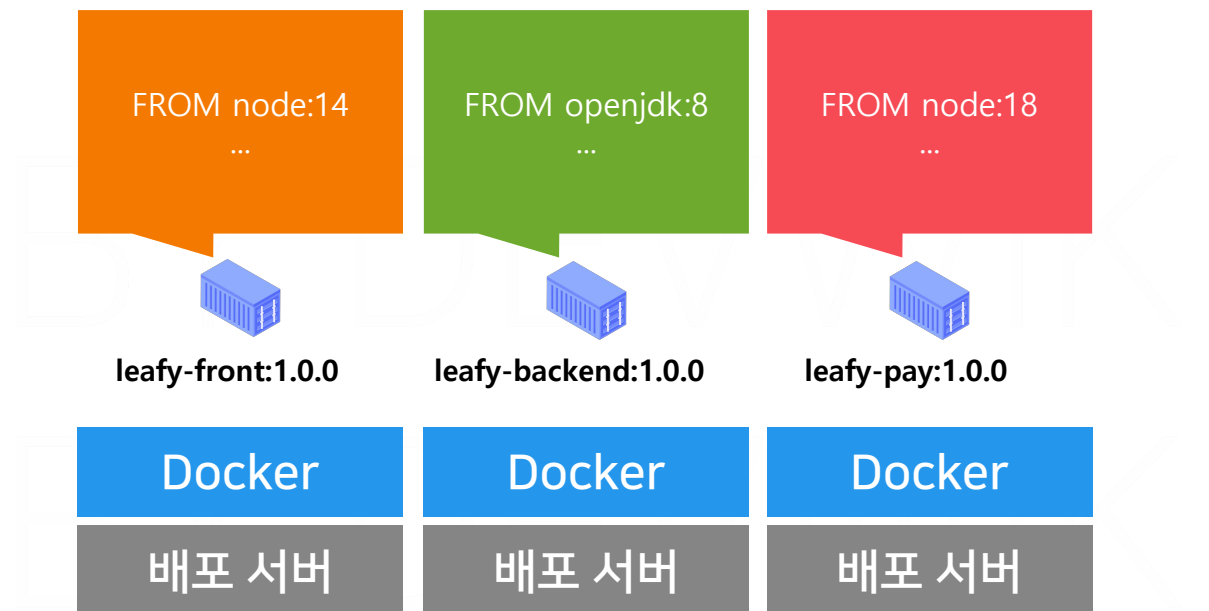
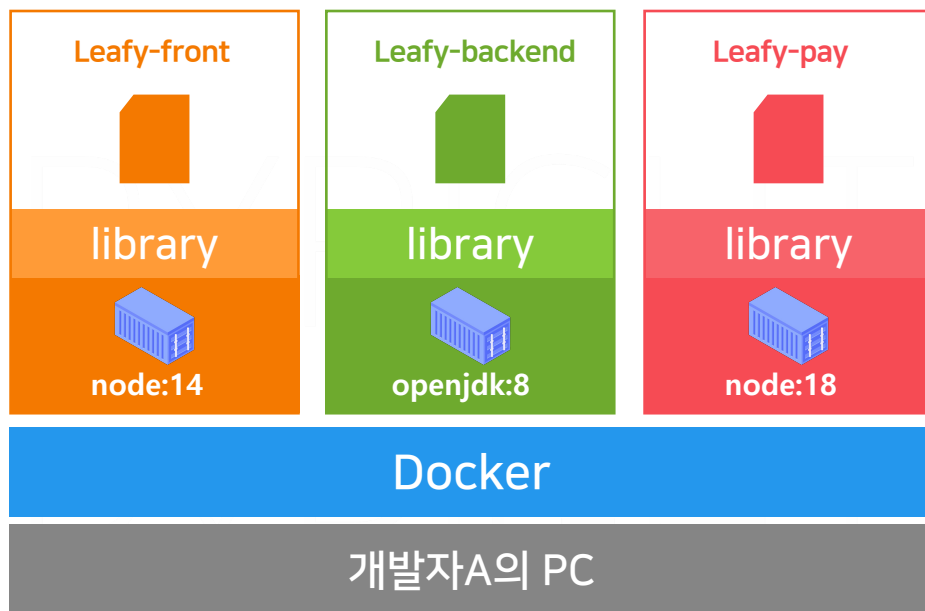


개발자 PC간 버전이 동일하지 않을 경우 빌드 및 실행 시 에러 발생

- 한 명의 개발자가 여러 개의 프로젝트를 개발할 경우, 각 프로젝트가 사용하는 언어나 라이브러리의 종류가 다를 수 있습니다. 여러 개의 환경을 개발자의 PC 한 대 에서 관리하기는 어렵습니다.
- 언어 및 라이브러리의 버전이 일치하지 않을 경우 “내 컴퓨터에서는 동작해” 와 같은 문제가 발생할 수 있습니다.



- 도커를 활용하면 개발자의 PC들과 배포 서버의 환경을 완전히 일치시킬 수 있습니다.
- IDE의 원격 개발 환경 기능을 통해 각 컨테이너별로 격리된 개발 환경을 구성할 수 있습니다.
- 개발자의 PC에 갖가지 라이브러리나 언어가 설치되지 않아 깔끔하게 유지할 수 있습니다.



leafy-frontend 애플리케이션 구동 시 리소스 사용량 제한

1. easydocker/leafy/leafy-backend 디렉터리 이동 및 소스코드 상태 변경

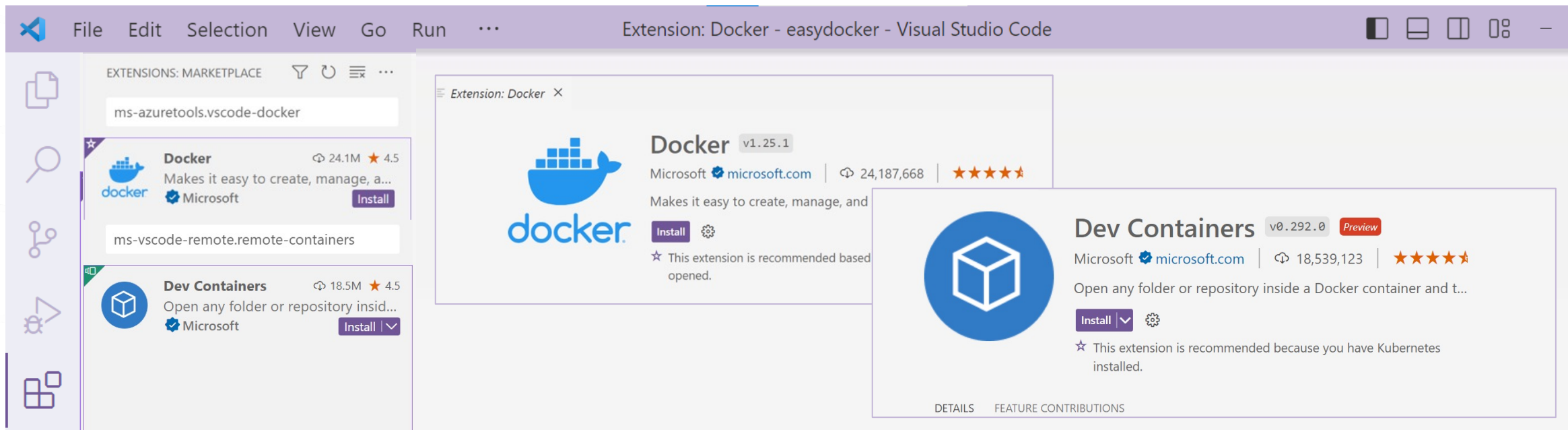
```
git reset --hard HEAD && git clean -fd
```

```
git switch 06-devcontainer
```

2. 로컬 머신에 node.js가 설치되어 있는지 확인

```
$ node --version
```

3. VSCode 실행 및 Extension 설치 (ms-azuretools.vscode-docker, ms-vscode-remote.remote-containers)



```
✓ leafy-frontend
  ✓ .devcontainer
    {} devcontainer.json
    🚢 Dockerfile
```

easydocker/leafy/leafy-frontend/.devcontainer/devcontainer.json

```
{
  "name": "Leafy-frontend project based node.js",
  "dockerFile": "Dockerfile",
  "forwardPorts": [80],

  "customizations": {
    "vscode": {
      "settings": {},
      "extensions": [
        "dbaeumer.vscode-eslint"
      ]
    }
  },

  "postCreateCommand": "npm install",
  "remoteUser": "node"
}
```

leafy-frontend
└─ .devcontainer
 ├─ {} devcontainer.json
 └─ Dockerfile

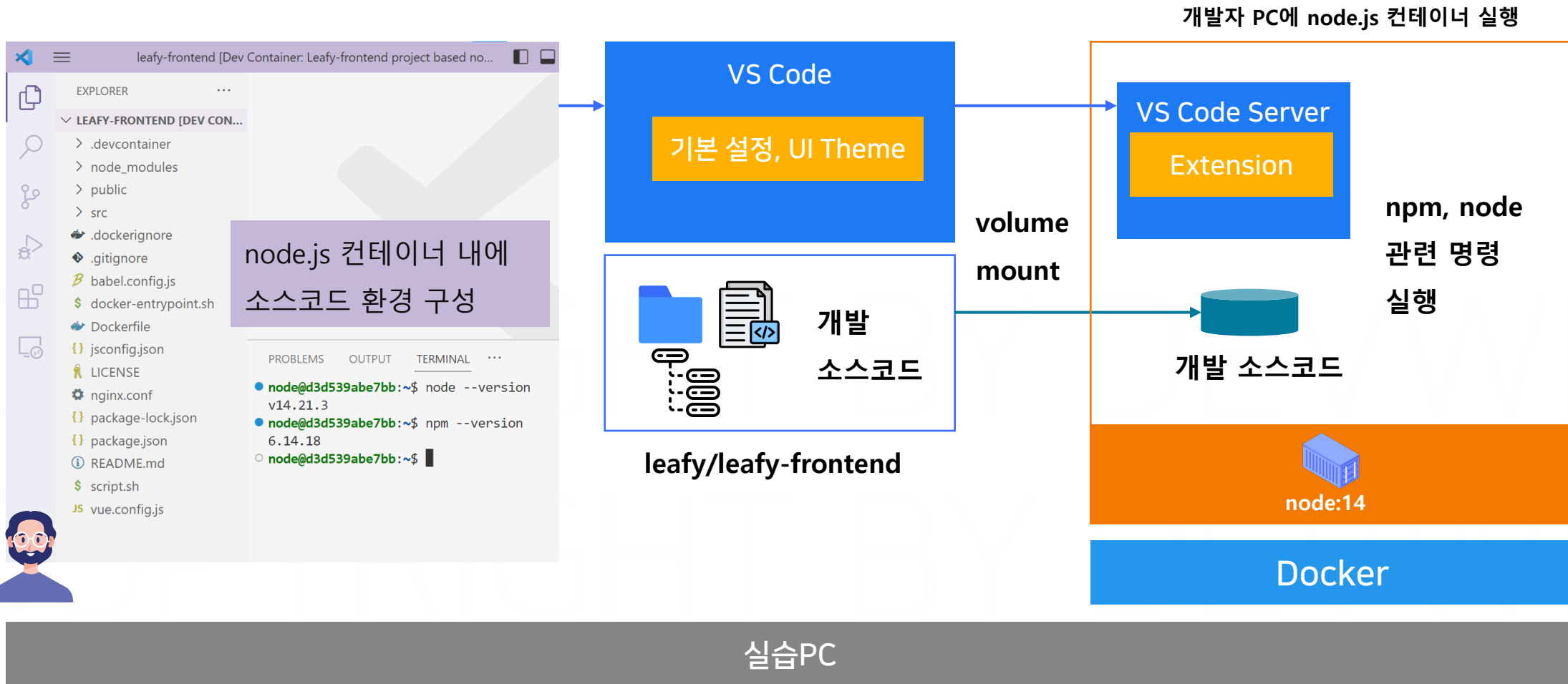
easydocker/leafy/leafy-frontend/.devcontainer/Dockerfile

```
FROM node:14

RUN apt update && apt install -y less man-db sudo

ARG USERNAME=node
RUN echo $USERNAME ALL=\(root\) NOPASSWD:ALL > /etc/sudoers.d/$USERNAME \
    && chmod 0440 /etc/sudoers.d/$USERNAME

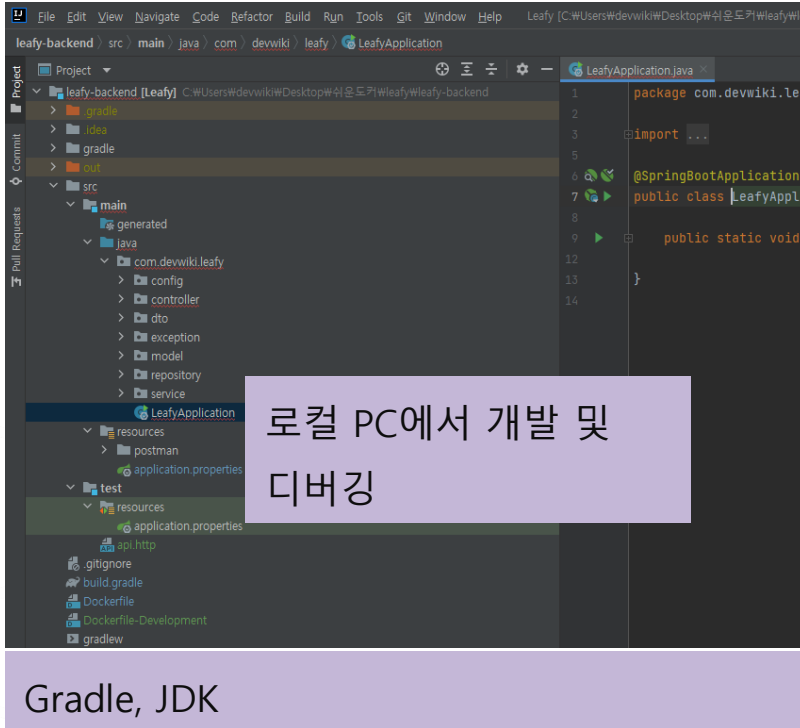
ENV DEVCONTAINER=true
```



로컬PC에 node.js 관련 환경이 없어도 개발 가능

PART8. 도커 실무

TOPIC 8. JAVA 개발환경 컨테이너 활용 (IntelliJ)



디버깅
:5005번 포트

테스트
:8080번 포트

개발자 PC의 소스로
localLeafyDevContainer 빌드 및 실행

실제 애플리케이션이 실행되는 환경

```
java -
agentlib:jdwp=transp
ort=dt_socket,server
=y,suspend=n,address
s=:5005
-jar leafy.jar
```



localLeafyDevContainer

Docker

실습PC

로컬 PC에서 실행되는 애플리케이션 환경을 컨테이너로 통일 가능